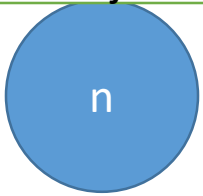


Eliminar un nodo n de un árbol

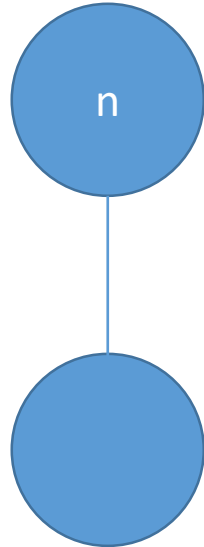
```
public void removeNode(Position<E> p) throws  
    InvalidPositionException{
```

n es raíz - casos

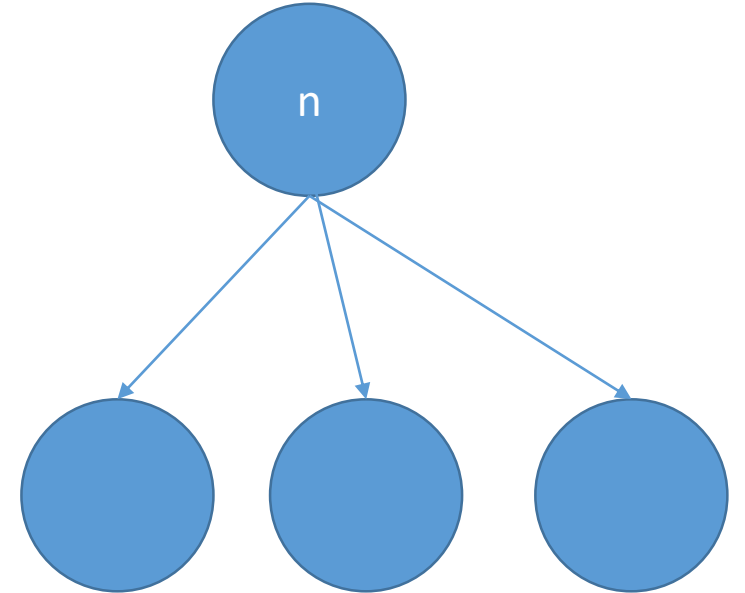
CASO 1: Eliminar el nodo raíz, el cual no tiene hijos.



CASO 2: Eliminar el nodo raíz, el cual tiene 1 hijo.



CASO 3: Eliminar el nodo raíz, el cual tiene más de un hijo.



n es raíz

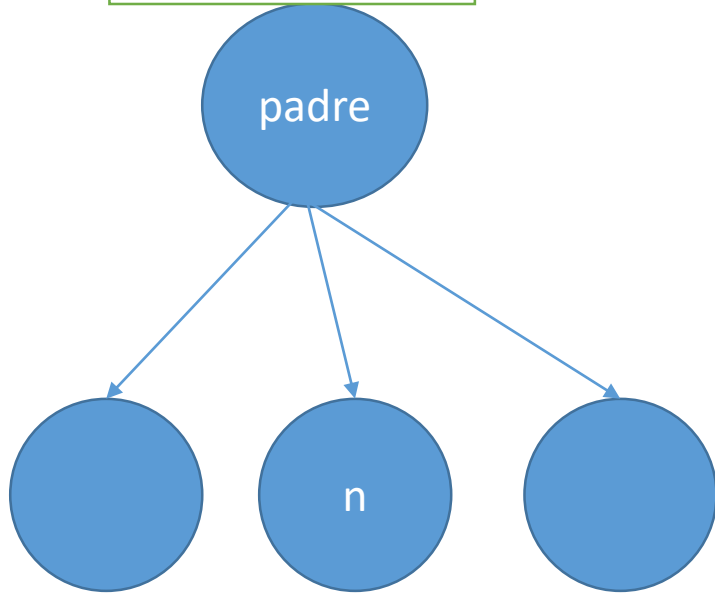
- Si no tiene hijos -> se elimina directamente el nodo. (root = null)
- Si tiene un solo hijo, este queda como nodo raíz.

```
if(root.getHijos().size()==1)//la raiz tiene un solo hijo
{
    Position<TNode<E>> rN=root.getHijos().first();
    rN.element().setPadre(null);
    root.getHijos().remove(rN);
    root= rN.element();
}
```

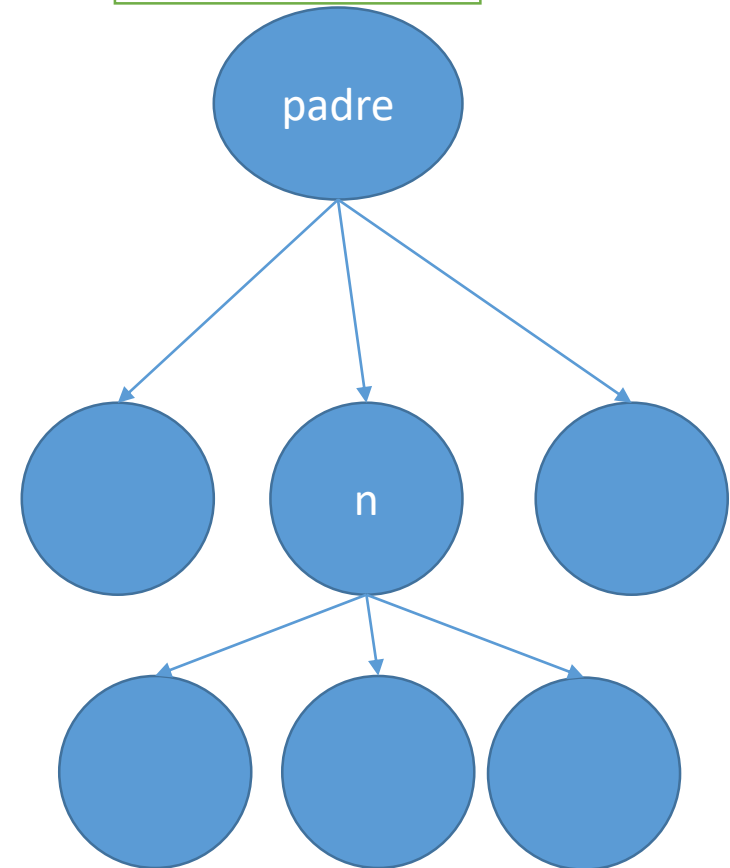
- Si tiene más de un hijo -> no se puede eliminar.
 - **throw new** InvalidPositionException("Solo se puede eliminar la raíz si es el único elemento o si tiene un solo hijo");

n es nodo interno u hoja

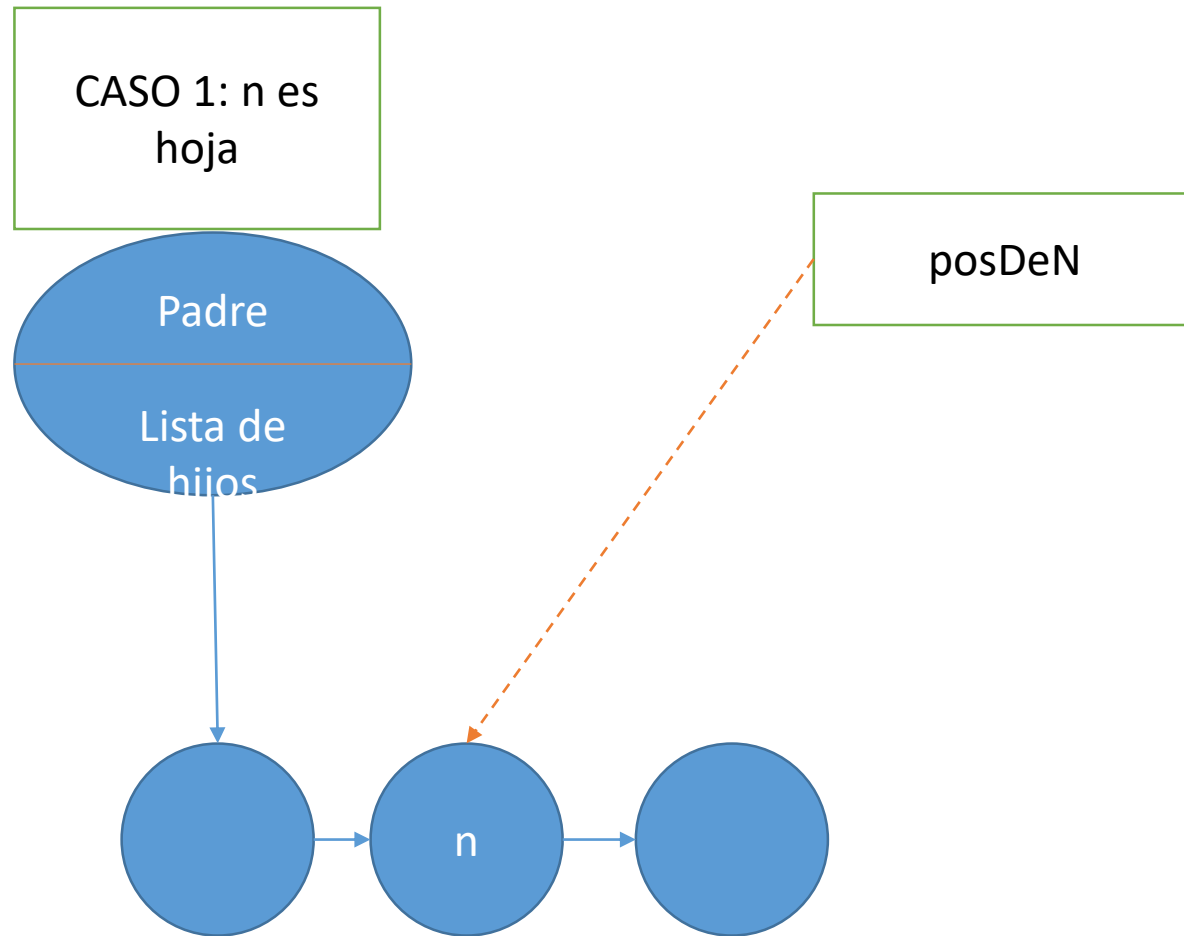
CASO 1: n es
hoja



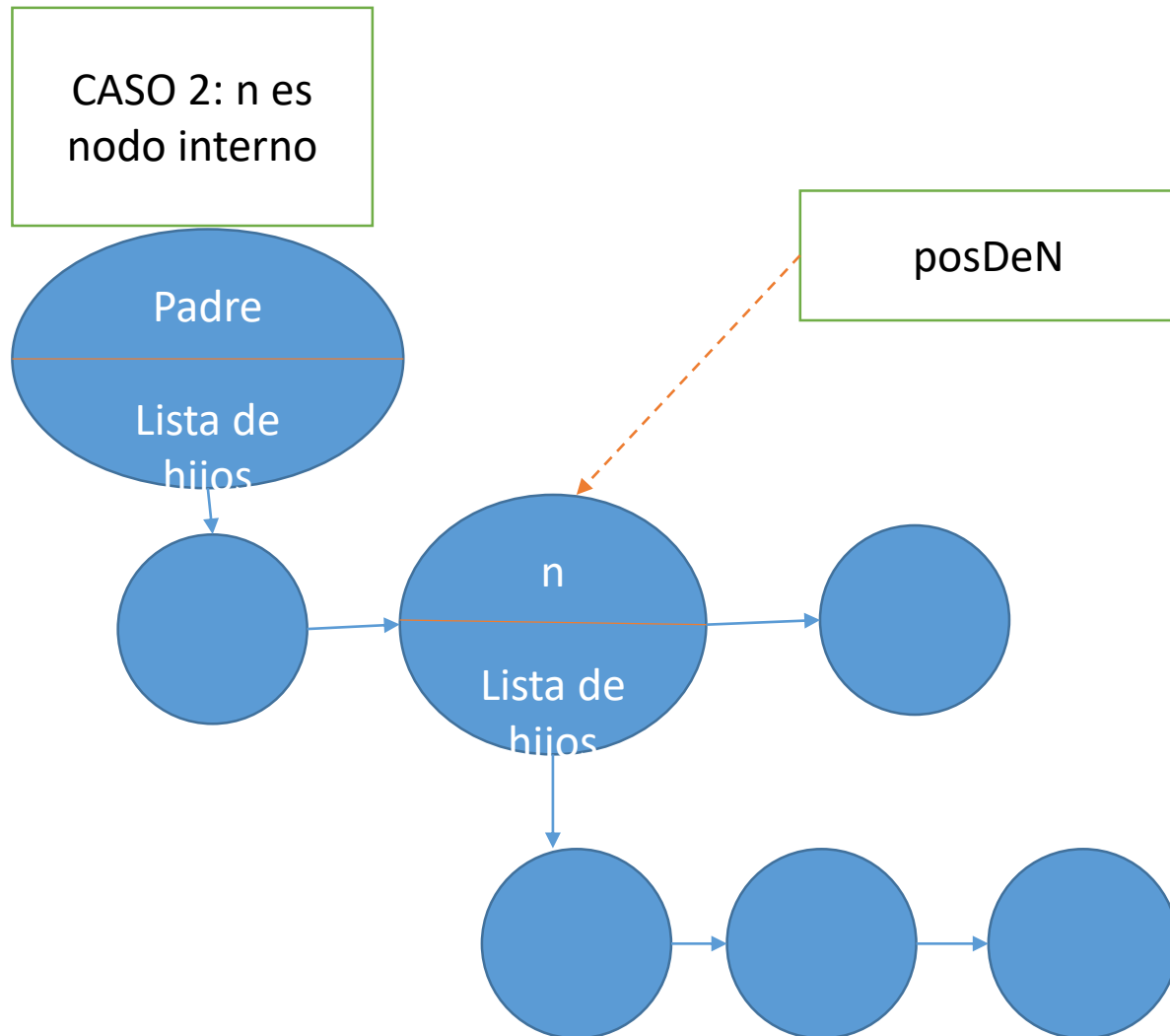
CASO 2: n es
nodo interno



Pensando en la estructura....



Pensando en la estructura....



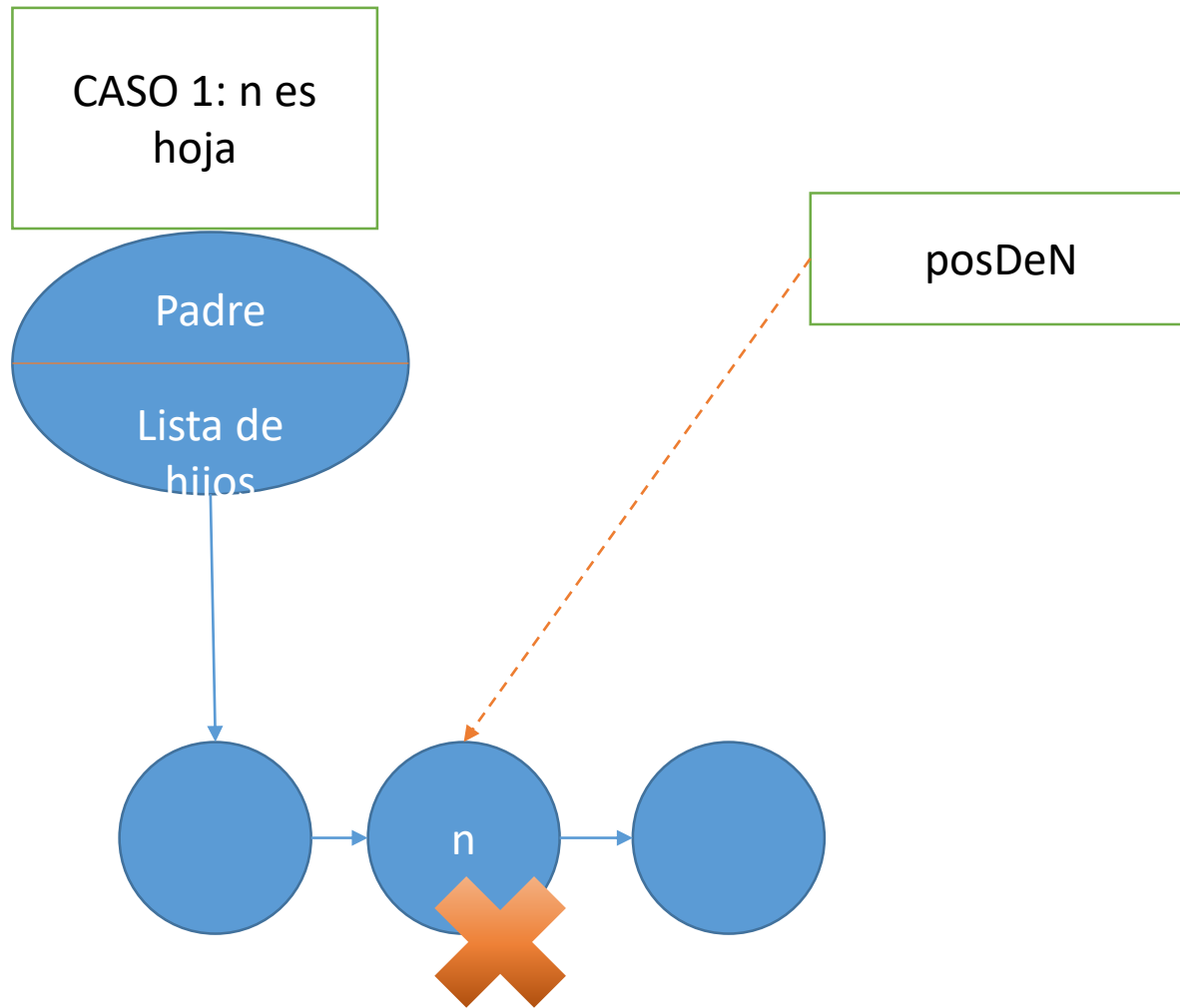
Busco la posición de n dentro de la lista de hijos del padre (posDeN)

- Obtener el padre de n : `TNodo<E> padre=n.padre();`
- Buscar a n en los hijos de padre, recorre la lista de hijos hasta encontrar uno cuyo elemento sea igual a n. Guardo en **posDeN** la posición de n en la lista de hijos.

```
PositionList<TNodo<E>> hPadre=padre.getHijos(); //hijos del padre (hermanos de n)
PositionList<TNodo<E>> hN=n.getHijos(); //hijos de n

//buscar a n dentro de los hijos del padre
Position<TNodo<E>> posDeN;
Position<TNodo<E>> cursor= hPadre.first();
while(cursor.element() != n && cursor != null){
    if (cursor == hPadre.last())
        cursor = null;
    else
        cursor = hPadre.next(cursor);
}
if(cursor != null)
    posDeN = cursor;
else
    throw new InvalidPositionException("La estructura no corresponde a un arbol valido")
```

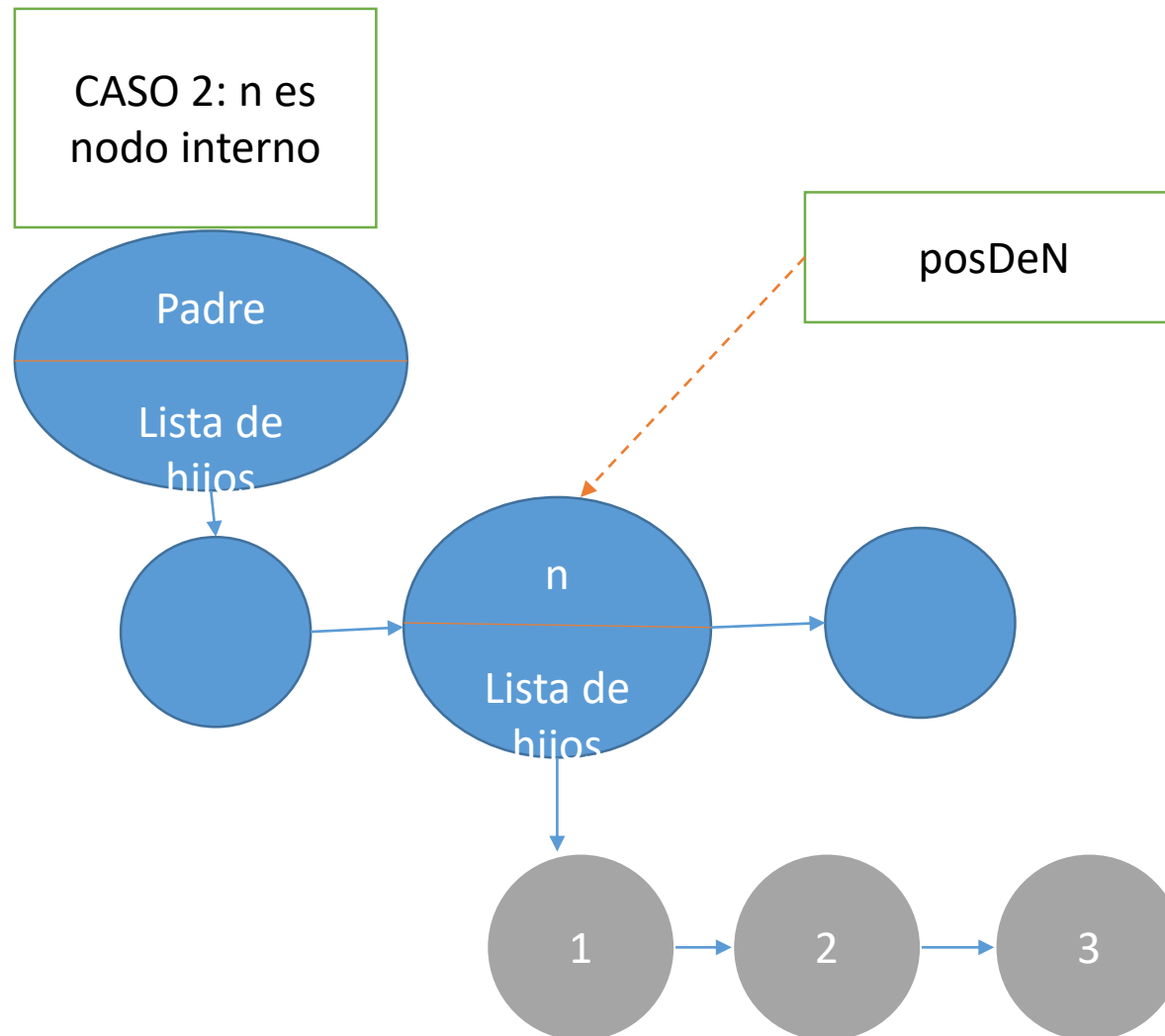
Pensando en la estructura....



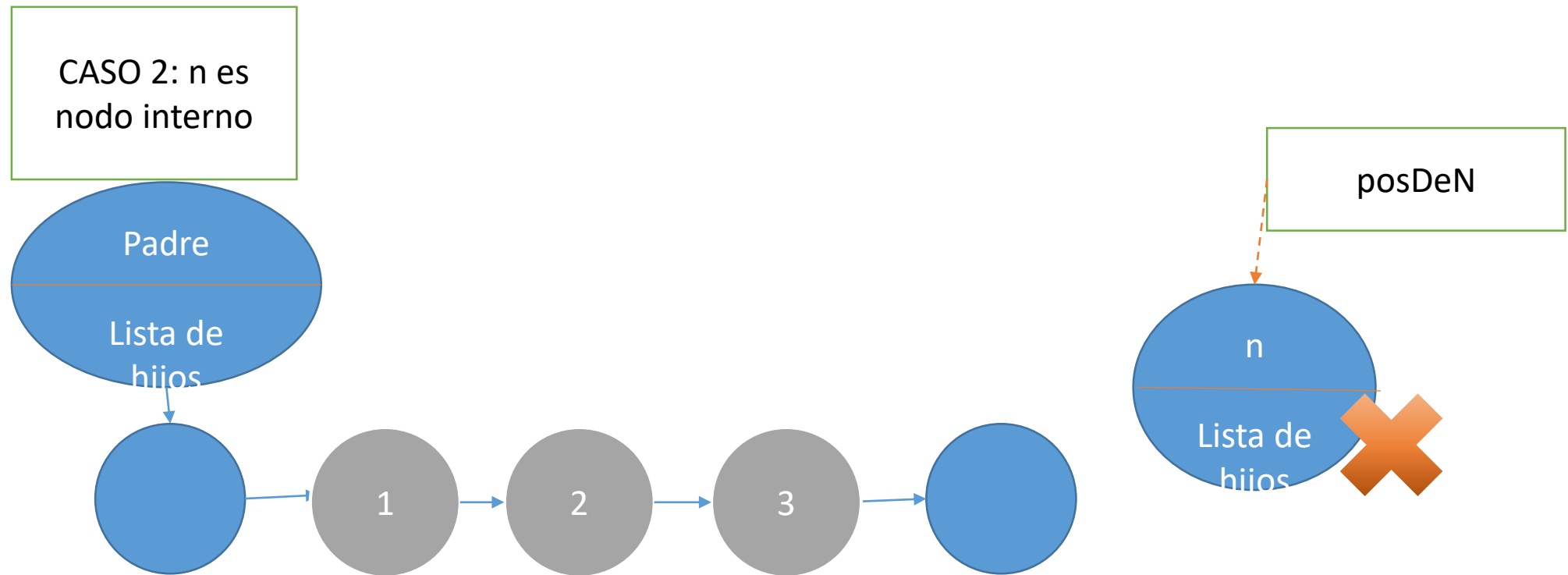
Caso 1: n es hoja

- Si n no tiene hijos se puede eliminar directamente de la lista de hijos de P.
- En el paso anterior guardamos en posDeN la posición del nodo, la cual vamos a utilizar para eliminar el nodo.
- hPadre.remove(**posDeN**);

Pensando en la estructura....



Pensando en la estructura....



Inserto los nodos que se encuentran en la lista de hijos en el lugar donde se encontraba n.

Caso 2: n es nodo interno

- Si n tiene hijos, insertar los hijos como hijos de P.
- `PositionList<TNodo<E>> hN=n.getHijos();`//obtenemos la lista de hijos de n

```
while (!hN.isEmpty())
{
    Position<TNodo<E>> hijoN=hN.first();
    hPadre.addBefore(posDeN,hijoN.element());
    hijoN.element().setPadre(padre);
    hN.remove(hijoN);
}
```